Ders 6 Genişletilmiş Notu: Hash Fonksiyonlarının Özellikleri, Uygulamaları ve Dinamik Programlamaya Giriş

Ders 6 Genişletilmiş Notu: Hash Fonksiyonlarının Özellikleri, Uygulamaları ve Dinamik Programlamaya Giriş

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Hash Fonksiyonlarına Uygulama Gözüyle Bakış
- 2. İstenen Özellikler (Desirable Properties)
- 3. Özellikler Arasındaki İlişkiler: İki Karşı Örnek
- 4. Uygulamalar ve Gereken Özellikler
- 5. Dinamik Programlamaya Giriş
- 6. Fibonacci Sayıları ve Memoization
- 7. En Uzun Palindrom Alt Dizisi
- 8. Çubuk Kesme (Rod Cutting) Problemi
- 9. Çubuk Kesmenin Naif Özyinelemeli Çözümü
- 10. Memoized ve Aşağıdan Yukarı Çubuk Kesme
- 11. Kesim Yerlerini Bulmak
- Örnek Sorular

1. Hash Fonksiyonlarına Uygulama Gözüyle Bakış

Hoca, önceki derslerde hash tablolarına bir giriş yapıldığını, ama hash fonksiyonlarının şifreleme (encryption) dahil pek çok uygulamada kullanılan, sürekli gelişen ve hakkında sürekli makale yayımlanan bir konu olduğunu hatırlattı. Bu dersin ilk kısmının amacı, hash fonksiyonlarının uygulamalarda önem kazanan özelliklerini isimleriyle tanıtmaktı: ileride bir araştırmada bu kavramlarla karşılaşıldığında akılda bir şeylerin canlanması isteniyordu. Kripto paralarla ilgilenenlerin bu kavramları zaten bilebileceğini, ama burada anlatılacak uygulamaların doğrudan kriptografik olmadığını, amacın bir "fikir jimnastiği" olduğunu söyledi. Anlatımın tamamı Microsoft Whiteboard üzerinde yapıldı.

1.1 Tanım

Tahtaya yazılan tanım şudur (ekrandan, 00:10:00):

$$h : \{0,1\}^* \to \{0,1\}^d$$

Girdi ikilidir; bir string verilse bile sonunda sıfır ve birlerden oluşur. Üstteki yıldız, girdinin kaç haneden oluştuğunun bilinmediğini gösterir. Hoca parola örneğini verdi: bir parola 5, 10 ya da 128 haneli olabilir; girdi boyutu değişkendir. Hash fonksiyonu bu değişken boyutlu girdiyi sabit boyutlu (fixed size) bir çıktıya, d bitlik bir hash değerine dönüştürür. Hoca önce "sabit çıktı" deyip "sabit boyutlu çıktı" diye düzeltti.

1.2 Motive Edici Senaryo

Bir dosya dijital imzayla imzalanmış ya da sunucuya yüklenmiş olabilir ve bunun gerçekten o dosya olup olmadığı doğrulanmak (verify) istenir. Dosya herkese açık (public) olduğu için kullanılan hash fonksiyonu da gizli değildir. Doğrulanmak istenen dosya m_1, ürettiği değer h_1 olsun. Kötü niyetli biri m_1 yerine öyle bir m_2 dosyası oluşturabilir ki $m_1 \neq m_2$ olduğu hâlde $h(m_1) = h(m_2)$ olur. Dosya ve parola doğrulama gibi uygulamalar için bu durumla baş edebilen fonksiyonlar üretmek gerekir: iki farklı girdi için farklı çıktı üretilmelidir. Bu, daha önce görülen çakışma (collision) kavramıdır ve önemli özelliklerden biridir.

1.3 Bilinen Algoritmalar ve Güvenlik Düzeyi

Hoca MD4, MD5, SHA-1 ve SHA-256'yı hash fonksiyonlarından yararlanan kriptografik algoritmalar olarak andı. Bu algoritmaların hepsi kaba kuvvetle (brute force) kırılmaya açıktır: sürekli denenirse bir çakışmaya rastlanabilir. Bir algoritmanın güvenliğini, bu denemelerin ne kadar sürdüğü belirler; "güvenlik düzeyi" (security level) bununla kastedilir. Hoca, bazı algoritmaların tasarlandıkları düzeyden çok daha az denemeyle kırılabildiğinin ispatlandığını, bunun sürekli gelişen bir alan olduğunu ve ilgilenenlerin makalelere bakabileceğini söyledi.

2. İstenen Özellikler (Desirable Properties)

Hoca özellikleri tahtaya numaralandırarak yazdı (ekrandan, 00:20:00 ve 00:30:00).

2.1 Tek Yönlülük (One-Way, OW)

Tahtadaki tanım: "Given $y \in \{0,1\}^d$, it is hard to find x such that $h(x) = y$." Yani bir çıktı (hash değeri) verildiğinde, bu çıktıyı üretecek girdiyi bulmak zor olmalıdır.

Hoca bunu tahtada iki okla gösterdi: x'ten $h(x)$'e geçiş kolay (easy) olmalı, geri dönüş zor (hard) olmalı. h'yi hesaplamak uzun sürmemelidir, çünkü bu da sistemi yavaşlatır. Tek yönlülük yüzde yüz garanti edilemeyebilir; fonksiyonun katmanları artırılarak mümkün olduğunca sağlanmaya çalışılır.

Örnek olarak derste şimdiye kadar kullanılan mod ve asal sayı tabanlı fonksiyonlar ele alındı: $h(x) = x \bmod p$ ve çıktı y olsun. Girdiyi bulmak isteyen biri modüler aritmetikten yararlanır:

$$x = y + a \cdot p$$

a'nın farklı değerleri tek tek denenerek, kaba kuvvetle girdi bulunabilir. Demek ki derste kullanılan bu fonksiyon tek yönlü değildir; tek yönlü olması için geri dönüşü zorlaştıracak adımlar eklenmelidir.

2.2 Çakışmaya Dayanıklılık (Collision Resistant, CR)

Tahtadaki tanım: "infeasible to find x, x', $x \neq x'$ and $h(x) = h(x')$." Birbirinden farklı ama aynı hash'i veren bir (x, x') çiftini bulmak uygulanabilir (feasible) olmamalıdır. Yani iki farklı girdi için aynı hash'in üretilme olasılığı düşük olmalı, böyle bir çiftin bulunması zorlaştırılmalıdır.

Hoca bu özelliğin her uygulamada aynı ağırlıkta olmadığını vurguladı. Örneğin bir programlama dili için sözlük (dictionary) ya da bir arama tablosu (lookup table) oluşturulacaksa SHA gibi çok karmaşık bir fonksiyon kullanılmak istenmez, çünkü hesaplama süresi güvenlikten önemli hâle gelir. Birkaç çakışma sorun olmaz, onlar bağlı listeyle (zincirleme) çözülür; önemli olan sonuca hızla ulaşmaktır.

> **Hocanın vurgusu:** Bütün bu özellikler uygulamaya bağlı olarak önem kazanır.

2.3 Hedefli Çakışmaya Dayanıklılık (Target Collision Resistant, TCR)

Bu özellik bir öncekine benzer, yine çakışmayla ilgilidir; fark, bir x'in önceden verilmiş olmasıdır. Verilen x ile aynı hash'i veren başka bir girdi bulmak zor olmalıdır. CR'de aynı değeri veren iki ayrı x aranırken, burada biri eldeyken diğeri aranır.

Hoca bu iki özelliğin birbirini kapsadığını söyledi: bir fonksiyon CR ise zaten TCR'dir. TCR, CR'nin daha zayıf (weak) bir sürümü gibidir. Tahtaya bu ilişki CR $\rightarrow$ TCR diye yazıldı (ekrandan, 00:30:00).

> **Ders sonrası not:** Tahtadaki TCR tanımında "infeasible given x, to find $x = x'$, $h(x) = h(x')$" yazıyor (ekrandan, 00:20:00). Doğrusu $x \neq x'$'tür: verilen x'ten farklı ve aynı hash'i veren bir x' bulmak zor olmalıdır. Hocanın sözlü anlatımı da bu yöndedir.

2.4 Sözde Rastgelelik (Pseudo-Random)

Bir fonksiyon üretildiğinde onun rastgele gibi görünmesi hedeflenir. Tamamen rastgele olması sağlanamadığı için sözde rastgele (pseudo-random) olması sağlanmaya çalışılır. Hoca, önceki derslerde hash fonksiyonlarının asal sayılarla üretilmesinin nedeninin de bu olduğunu, fonksiyonun görünüşte rastgele olmasının istendiğini hatırlattı. Bu özellik yalnız ismen yazıldı.

2.5 Biçimlendirilemezlik (Non-Malleability, NM)

Tahtadaki tanım: "Given $h(x)$, to produce $h(x')$ where x and x' are related, e.g. $x' = x + 1$" (ekrandan, 00:30:00). Elde yalnız $h(x)$ varken, x ile gözlemlenebilir, kanıtlanabilir bir ilişkisi olan bir x' için biraz hesap yaparak $h(x')$ üretmek mümkün olmamalıdır. Biçimlendirilebilirlik (malleability) bunun mümkün olmasıdır; biçimlendirilemez fonksiyonlar bundan korur. Örneğin girdiler x ve $x + 1$ gibi ilişkiliyse ve $h(x)$ verilmişse, bu bilgilerden yola çıkarak $h(x + 1)$ üretilememelidir. Hoca, bu özelliğin uygulama örneği verilince daha iyi oturacağını söyledi.

3. Özellikler Arasındaki İlişkiler: İki Karşı Örnek

Hoca, CR ile TCR arasındaki kapsama ilişkisini hatırlattıktan sonra benzer bir ilişkinin başka özellikler arasında kurulup kurulamayacağını sordu: örneğin çakışmaya dayanıklı her fonksiyon aynı zamanda tek yönlü müdür? Bunu incelemek için iki yapı kurdu. İkisinde de başlangıç noktası, hem tek yönlü hem TCR olduğu varsayılan bir h fonksiyonudur: girdileri $x_1, x_2, \ldots, x_n$ bitleri, çıktısı d bitlik bir değerdir (ekrandan, 00:30:00).

Hoca bu anlatımın güvenli bir yapı önermek için değil, böyle bir şeyin yapılabildiğini göstermek ve kafada bir şeylerin oturması için verildiğini belirtti.

3.1 Tek Yönlülük Kalsın, Çakışmaya Dayanıklılık Bozulsun

Hoca önce şunu sordu: h'nin girdilerine ya da çıktısına bir şeyler eklenip fonksiyon daha karmaşık hâle getirilirse tek yönlülük bozulur mu? Cevap hayırdır; tek yönlülük karmaşıklıkla birlikte artan bir özelliktir. Daha önce mod üstüne mod alınması da hem çakışmaya dayanıklılığı hem tek yönlülüğü artırmak içindi; katman eklemek tek yönlülüğün önüne geçmez.

Bundan yola çıkarak girdilerden birine bir XOR kapısı eklendi: x_1 artık iki yeni bitin, a ile b'nin XOR'u olur. Yeni fonksiyon (ekrandan, 00:30:00 ve 00:36:30):

$$h'(a, b, x_2, x_3, \ldots, x_n) = h(a \oplus b, x_2, x_3, \ldots, x_n)$$

Yeni fonksiyon a, b, $x_2, \ldots, x_n$'yi alır; önceki fonksiyon x_1 alıyordu, şimdi onun yerine a ile b'nin XOR'unu alır. Hoca tahtadaki x_2, x_3 yazılarının üs değil indis olduğunu, altlarına yazılması gerektiğini de düzeltti.

- **Tek yönlü mü?** Evet. Diğer bitler değişmeden kaldığı ve fonksiyon geri kalan kısım için aynı sonucu verdiği için, başa bir şey eklemek tek yönlülüğü etkilemez. Bilgiye iki bit eklenmiştir ama bu tek yönlülüğü bozan bir şey değildir.

- **Çakışmaya dayanıklı mı?** Hayır. XOR iki bitin aynı olup olmadığına bakar. $a = 1, b = 1$ ile $a = 0, b = 0$ aynı XOR değerini (0) verir. Bu yüzden diğer bitler aynıyken

$$h'(a = 1, b = 1, x_2, \dots, x_n) = h(a = 0, b = 0, x_2, \dots, x_n)$$

olur (ekrandan, 00:36:30). Aynı hash'i veren farklı girdi çiftleri bolca bulunur. Tek yönlülük bozulmadı ama çakışmaya dayanıklılık bozuldu. Hoca sohbet penceresinden gelen, kendisini teyit eden cevapları da okudu.

3.2 Tek Yönlülük Bozulsun, TCR Kalsın

Sonra tersi soruldu: aynı h'den yararlanarak tek yönlülüğü bozan ama TCR'yi bozmayan bir fonksiyon tasarlanabilir mi? Hoca öğrencilere düşünme süresi verdi, sonra şu yapıyı tahtaya yazdı (ekrandan, 00:36:30):

$$h''(x) = \begin{cases} 0 \,\|\, x & |x| \leq n \\ 1 \,\|\, h(x) & \text{aksi hâlde} \end{cases}$$

Burada n, örneğin bir parolanın alabileceği en büyük uzunluk gibi düşünülebilecek bir değerdir. Girdinin uzunluğu n'den küçükse başına 0 eklenir ve girdi olduğu gibi hash diye dışarı verilir. Hoca, bir hash fonksiyonunun illa asal sayılarla mod alma işlemlerinden geçmesi gerekmediğini, bunun da tanımlanabilecek bir fonksiyon olduğunu söyledi. Girdi n'den küçük değilse h'nin çıktısının başına 1 eklenir.

- **Tek yönlülük bozuldu mu?** Evet. Kısa girdiler için çıktı, başında 0 olan girdinin kendisidir. Kötü niyetli biri baştaki sıfırı silerek x'i elde edebilir.
- **TCR bozuldu mu?** Hayır. h zaten TCR'dir. Çıktının başına 1 eklemek iki ayrı girdi için aynı hash'i üretmez; başında 1 olan çıktılar h'nin ürettiği farklı değerlerdir. Kısa girdiler ise aynı değere sahip olmadıkları için farklı çıktılar üretir. Böylece tek yönlülüğü bozulmuş ama TCR'si bozulmamış bir fonksiyon elde edilir.

Bu iki örnek, OW ile CR/TCR arasında CR ile TCR arasındaki gibi bir kapsama ilişkisi olmadığını gösterir: biri bozulurken diğeri korunabilir.

> **Ek bilgi (derste anlatılmadı):** h'''de başı 0 olan çıktılar ile başı 1 olan çıktılar ilk bitlerinden dolayı hiçbir zaman eşit olamaz. Bu yüzden h'' için bir çakışma ancak iki uzun girdi arasında olabilir; bu da doğrudan h'nin bir çakışmasıdır. Literatürde bu tür yapılar, "çakışmaya dayanıklılık tek yönlülüğü gerektirmez" sonucunu göstermek için büyük olasılıkla ders kitaplarında da kullanılan standart karşı örneklerdir.

4. Uygulamalar ve Gereken Özellikler

Hoca, her uygulama için hangi özelliklerin önemli olduğunu öğrencilere sorarak ilerledi (tahta başlığı "APPLICATIONS", ekrandan, 00:40:00).

4.1 Parola Saklama (Password Storage)

Bir sisteme parolayla giriş (login) sağlayan bir uygulama düşünülsün. Giren kişinin parolasını kontrol etmek için parola bir yerde saklanmalıdır. Parolanın kendisi yerine hash'i saklanır: tahtada "Store $h(PW)$, not PW on computer" (ekrandan, 00:50:00). Biri bu dosyayı ele geçirirse parolaları değil hash'leri ele geçirmiş olur. Hoca, kullanıcının parolasını kaybetmesi durumunda bu sistemin işe yaramayacağını, onun için farklı yöntemler gerektiğini; burada yalnız içeri girmeyi sağlayan sistemin konuşulduğunu belirtti.

Soru: Bu sistem için OW, CR, TCR ve NM'den hangisi en önemlidir? **Cevap:** Öğrenciler tek yönlülük dedi; hoca onayladı. Kötü niyetli biri saklanan hash'i ele geçirebilir, ama hash'ten parolaya hiçbir şekilde geri dönülmemelidir.

Soru: Bu örnekte CR ve TCR'nin önemi nedir; sağlanmazsa nasıl bir sorun çıkar? **Cevap:** Bir öğrenci "başka bir parolayla da girilebilir" dedi. Hoca bunların da önemli ve gerekli olduğunu, ama parola yeterince uzunsa çakışma ihtimalinin zaten düşük olduğunu söyledi. Ayrıca kaba

kuvvet denemelerine karşı, örneğin üç başarısız denemeden sonra kullanıcı sistemden atılabilir. Bu yüzden parola saklamada asıl kritik olan tek yönlülüktür. Tahtada OW daire içine alınmış, altında CR ve TCR yazılıdır.

Bu noktada bir öğrenciyle uzunca bir tartışma oldu:

Soru: Parola uzunsa çakışma olmaz dendi; ama "A" harfinin hash'i ile çok uzun bir parolanın hash'inin aynı olmayacağının garantisi yok. Çakışma parolanın uzunluğuna bağlı değil, değil mi? **Cevap:** Hoca, parolanın uzun olmasının entropisinin yüksek olması demek olduğunu söyledi; örneğin 128 bitlik bir parola. Daha yüksek entropili bir parola için üretilen değerlerde sözde rastgeleliği yakalamak kolaylaşır ve çakışma riski azalır. Kaba kuvvetle tarayan biri için 10 haneli bir parolayı bulmak 20 haneli bir parolayı bulmaktan kolaydır. Bununla birlikte hoca öğrencinin haklı olduğunu kabul etti: hash fonksiyonu öyle yazılabilir ki tek karakterlik bir parola ile on karakterlik bir parola aynı hash'i üretir; bu tamamen fonksiyonun tasarımına bağlıdır. Fonksiyonlar genelde üretilen değerlerin çeşitliliği parola uzunluğuyla orantılı artacak şekilde tasarlanır.

Soru: Parolanın uzunluğunun hash değerinin karmaşıklığına etkisi bir fonksiyonda nasıl görülebilir? Kısa parolada basit, uzun parolada karmaşık hash üreten bir fonksiyon anlamsız gibi geliyor. **Cevap:** Hoca, üretilen hash'in hem kısa hem uzun parola için "anlamsız" olduğunu söyledi ve konuyu güncel hash fonksiyonları ve sözde rastgelelik üzerinden bir örnekle, yazarak açıklamayı sonraki derse bıraktı.

4.2 Dosya Değişikliği Algılayıcı (File Modification Detector)

Tahtadaki tanım: "For each file, store $h(F)$ in a secure [place]; check if F is modified by recomputing $h(F)$" (ekrandan, 00:50:00). Her dosya için bir hash üretilir ve güvenli bir yerde saklanır. Dosyanın kendisi güvenli bir yerde değildir, ama hash güvenlidir. Karşı taraf dosyayı aldığında aynı dosya için hash'i yeniden hesaplar ve iki hash'i karşılaştırarak dosyanın değiştirilip değiştirilmediğini anlar. Değiştirilmiş bir F' için beklenen $h(F) \neq h(F')$'tür.

Soru: Bu uygulamayı başarılı bir şekilde kırmak (break) için ne yapılır? **Cevap:** Kötü niyetli biri F'yi öyle değiştirir ki iki hash hâlâ aynı kalır. Dosya değişmiştir ama değiştiği anlaşılmaz ve güncel dosya üzerinden işlem yapılır. Hocanın örneği: birine belli bir miktar ödemeyi taahhüt eden bir belge hashlenir, sonra belgedeki fiyat birdenbire artırılır; hash'ler aynı olduğu için itiraz edilemez.

Soru: Bu uygulamada hangi özellik önemlidir; örneğin tek yönlülük önemli mi? **Cevap:** Tek yönlülük burada önemli değildir; dosya zaten açıktır ve karşı tarafa gönderilmektedir. Önemli olan, aynı hash'i üreten başka bir dosyanın üretilememesidir; hoca bunu "collision resistance" diye adlandırdı. Tahtada OW'nin altı çizilmiş (önemsiz), yanında TCR yazılmıştır (ekrandan, 00:50:00 ve 01:00:00).

> **Ek bilgi (derste anlatılmadı):** Dosya değişikliği senaryosunda saldırganın elinde belirli bir F vardır ve ona eş bir F' aramaktadır; bu tam olarak TCR'nin (ikinci ön görüntü direnci, second-preimage resistance) tanımıdır. Bu yüzden tahtadaki TCR ifadesi bu uygulama için daha kesin bir adlandırmadır; CR ondan güçlü olduğu için onu da sağlar.

4.3 Dijital İmza (Digital Signature)

Hoca bu konunun kriptoloji dersiyle kesiştiğini, hash fonksiyonlarına bu yüzden ara ara değindiğini söyledi.

Dijital imzada iki anahtar vardır: açık anahtar (public key, PK) ve özel anahtar (private key, SK) (ekrandan, 01:00:00). İşlem adımları:

- **İmzalama (signing):** $\sigma = \text{sign}(SK, M)$. Kişinin özel anahtarı ve imzalanacak belge M kullanılarak bir imza σ üretilir. İmza karşı tarafa gider; imzalanan belge de onunla birlikte gider.
- **Doğrulama (verification):** $\text{verify}(M, \sigma, PK)$. Belge, imza ve kişinin açık anahtarı kullanılır; sonuç doğru ya da yanlıştır (T/F): bu kişi bu belgeyi imzalamıştır ya da imzalamamıştır.

Özel anahtar ile açık anahtar birbiri hakkında doğrudan bilgi açığa çıkarmamalı, ama matematiksel olarak da ilişkili olmalıdır. Bunun nasıl sağlandığı kriptoloji dersinin konusudur.

Soru: Böyle bir sistem için hangi özellikler önemlidir? **Cevap:** Burada dikkat edilmesi gereken, imzalanan belge ile hash arasındaki ilişkidir. Kötü niyetli birine belge ve doğrulama için gelen hash verilmiş olsun. A kişisi bir belgeyi imzalamışken, B kişisi A'nın başka bir belgeyi imzaladığını düşünürse bu kötü olur. Saldırgan, imzalanan belgenin başka bir belge olduğunu bir şekilde gösterebilirse girişimi başarılı olur. Bu yüzden en önemli özellik yine çakışmaya dayanıklılıktır: saldırgan, elindeki belge M için $h(M) = h(M')$ eşitliğini sağlayacak başka bir M' arar. Tahtada TCR daire içine alınmış ve $h(M) = h(M')$ yazılmıştır (ekrandan, 01:00:00).

Soru: Biçimlendirilemezlik burada önemli mi? **Cevap:** İmzalanan belgeler arasında x ve $x+1$ gibi matematiksel bir ilişki yoksa önemli değildir. Birbirinden biraz fazla ya da biraz eksik olabilecek sayılar değil de metin belgeleri söz konusuysa NM önem kazanmaz.

Soru: Anahtarlar için önemli olmaz mı; bir özel anahtara karşılık birden fazla açık anahtar olsa? **Cevap:** Doğrulama işleminde özel anahtar yer almaz. Saldırganın elinde doğrulama için açık anahtar zaten vardır; açık anahtardan özel anahtara geri dönülebilirse bu kötü olur. İki anahtar arasındaki ilişki geri döndürülemez olmalıdır; ama bu ilişkinin nasıl saklandığı bir hash fonksiyonuyla tutulmayabilir ve bu dersin konusu değildir.

Soru: Kastedilen, özel anahtarla bir şey imzalandığında başka bir açık anahtarla başkasının hak iddia etmesiydi. **Cevap:** Hoca soruyu, saldırganın aynı belgeyi aslında başkasının imzaladığını iddia etmesi olarak yorumladı. Bu da farklı girdiler için aynı hash'in üretildiği bir durumdur; burada değişen girdi belge değil anahtardır. Sonuçta yine çakışmaya dayanıklılık gerekir. Öğrenci de bunu onayladı.

4.4 Taahhüt (Commitment)

Dördüncü uygulama taahhüt (commitment) sistemleridir: ihaleler, kimsenin teklifinin açık olmadığı sanat müzayedeleri gibi. Açık artırmanın tersine burada kapalı zarf usulü ihale söz konusudur.

Tahtadaki kurgu (ekrandan, 01:07:20):

- A'nın bir x teklifi vardır ("A has x").
- A, $C(x)$'i hesaplar ve B'ye gönderir ("A computes $C(x)$ and submits it to B"). B, ihaleyi düzenleyen kuruluş ya da sanat galerisidir.
- B yalnız $C(x)$'i görür. Bütün teklifler toplanıp süre bitince A, $C(x)$'i açarak x'i gösterir ("A opens $C(x)$ to reveal x") ve en yüksek teklifi veren kazanır.

Teklifin kendisi değil hash'i gönderilir, çünkü teklifler süre bitmeden açılmaz; biri diğerinin teklifini görüp artırmasın diye. $C(x)$ herkese açıktır: zarfın dışını A da B de görür, zarflar toplanır, ama zarfın içi bilinmemelidir. Tahtada iki şart yazılıdır:

- 1) $C(x)$, x'i açığa çıkaramamalıdır (It can not reveal x).
- 2) Sistemi A'ya karşı da korumalıdır (It can also protect the system from A). Teklif veren kişinin dürüstlüğü kontrol edilemeyeceği için onun niyetine karşı da önlem alınmalıdır. Hocanın örneği: biri 20 milyon teklif eder, sonra işi 15 milyona da yapabileceğini görüp "aslında 15 demiştim" diyememelidir.

Soru: Bu uygulamada hangi özellikler önemlidir? **Cevap:** Hoca tek tek ilerledi (tahtada OW ve CR işaretli, altında NM yazılı):

- **Tek yönlülük önemlidir:** $C(x)$, x'i açığa çıkarmamalıdır. Zarf ortada dolaşırken içindeki x'e kimse ulaşmamalıdır.
- **Çakışmaya dayanıklılık önemlidir:** teklif veren kötü niyetliyse eski teklif yerine aynı hash'i veren başka bir teklif verdiğini iddia edebilir. Aynı hash'i veren daha düşük ve daha yüksek iki teklif bulunamamalıdır.
- **Biçimlendirilemezlik de önemlidir:** zarfların içindeki bilgiler fiyat olduğu için birbiriyle matematiksel olarak ilişkilidir. $C(x)$ verildiğinde $C(x+1)$ üretilememelidir. Aksi hâlde biri başkasının teklifinin bir fazlasını üretip gönderir ve çok para kaybetmeden ihaleyi kazanır.

Hoca, bu özelliklerin isimleriyle internette araştırıldığında onlarca algoritmayla birlikte karşılaşılacağını, ilgi duyanların detaylı araştırabileceğini söyleyerek konuyu kapattı.

5. Dinamik Programlamaya Giriş

Aradan sonra dinamik programlamaya (dynamic programming) geçildi; bu kısım PowerPoint slaytı (week6) ve tahtayla anlatıldı.

Dinamik programlama da bir problemi alt problemler üzerinden çözmek için ortaya atılmış bir yöntemdir. "Programlama" kelimesi bilinen anlamda kod yazmak değildir; optimizasyona yönelik bir çıkarım yapılır. Böl ve fethet (divide and conquer) algoritmaları da problemi alt problemlere ayırır, ama çoğu zaman gerektiğinden fazla alt problem çözer: birbiriyle örtüşen (overlap eden) alt problemler çıkar ve bunlar tekrar tekrar çözülür. Dinamik programlamanın amacı her alt problemi mümkünse yalnız bir kez çözmek, aynı sonuçları tekrar hesaplamanın getireceği zamandan tasarruf etmektir. Daha çok optimizasyon problemlerinde kullanılır.

Slayttaki dört adım (ekrandan, 01:20:00):

- 1. Alt problemleri tanımla (define sub problems): problemdeki alt problemler nelerdir?
- 2. Alt problem çözümlerini ilişkilendir (relate subproblem solutions): aralarındaki matematiksel ilişkiyi kur.
- 3. Özyinele ve hatırla (recurse & memorize) ya da DP tablosunu aşağıdan yukarı kur (build DP table bottom up).
- 4. Asıl problemi çöz (solve original problem).

Dinamik programlama hem özyinelemeli (recursive) yöntemlerle hem de aşağıdan yukarı (bottom-up), yani klasik döngülerle yapılabilir. Önemli olan, alt problemlerde tekrar eden gereksiz hesaplamalardan kaçınmaktır. Bunun için hesaplanan değerler bir yerde saklanır: özyinelemeli fonksiyonlarda bu işe memoization denir; aşağıdan yukarı yazılan çözümlerde DP tabloları kurularak alt problemlerin çözümleri kaydedilir.

> **Ders sonrası not:** Hoca ve slayt bu kavrama "memorization" (memorize) diyor. Literatürdeki terim memoization'dır (hesaplanan sonucu bir "memo"ya, nota yazmak); anlam aynıdır.

6. Fibonacci Sayıları ve Memoization

Hoca konuya hızlı bir hatırlatma olarak Fibonacci sayılarıyla girdi. $F_1 = F_2 = 1$'dir; sonraki her sayı önceki ikisinin toplamıdır.

6.1 Naif Özyineleme

Naif özyinelemeli Fibonacci algoritmasının analizi daha önce, bir ağaç üzerinden yapılmıştı. F_n için F_{n-1} ve F_{n-2} üretilir; F_{n-1} için F_{n-2} ve F_{n-3}; F_{n-2} için F_{n-3} ve F_{n-4} üretilir ve bu böyle gider (ekrandan, 01:25:05). F_{n-2} ve F_{n-3} gibi değerler tekrar tekrar bulunur ve bu zaman kaybettirir.

Tahtadaki üst sınır:

$$T(n) = \sum_{i=0}^{n-1} 2^i = O(2^n)$$

Hoca, ağacın her yaprağı aynı seviyede bitmediği için daha önce gerçek değerin altın oran (golden ratio) üzeri n olduğunu gösterdiğini hatırlattı.

> **Ders sonrası not:** Hoca altın oran üzeri n'i "ortalama" (average) durum olarak andı. Algoritma deterministtir; büyük olasılıkla kastedilen, çalışma süresinin 2^n üst sınırından daha sıkı olarak $\Theta(\varphi^n)$, $\varphi \approx 1{,}618$ olmasıdır.

6.2 Memoized DP Algoritması

Tahtadaki kod (ekrandan, 01:25:05):

```
fib(n):
    if n in memo: return memo[n]
    if n <= 2: f = 1
    else f = fib(n-1) + fib(n-2)
    memo[n] = f
    return f
```

Başta bir kontrol yapılır: aranan n. Fibonacci sayısı daha önce bulunup saklandıysa doğrudan döndürülür, daha fazla ilerlenmez. $n \leq 2$ ise değer 1'dir. Değilse özyineleme işletilir ve bulunan değer tekrar kullanılabilsin diye saklanır (store). Hoca kodda "if" yazdığı bir yerin "f" olması gerektiğini düzeltti. Bu, dinamik programlamanın görülebilecek en basit örneğidir.

6.3 Dinamik Programlamada Karmaşıklık

Dinamik programlamayla çözülen bir problemin karmaşıklığı temel olarak şöyle hesaplanır (ekrandan, 01:42:10):

$$\text{Süre} = \#\text{alt problem} \times \text{her alt problemi çözme süresi}$$

Kaç kez özyinelemeye girildiği ve her seferinde ne iş yapıldığının çarpımı toplam işlem sayısını verir. Hoca önce "recursion ve memoization maliyetlerinin toplamı" deyip ardından çarpım biçimini yazdı.

Fibonacci'de özyinelemeye n kez girilir, çünkü her sayı yalnız bir kez görülür. Saklama ve toplama maliyeti sabittir, $O(1)$. Bu yüzden memoized Fibonacci $\Theta(n)$ sürer. Önceki üst sınır 2^n, sıkı sınır altın oran üzeri n iken n'e inmek ciddi bir tasarruftur.

7. En Uzun Palindrom Alt Dizisi

7.1 Problem

En uzun palindrom alt dizisi (longest palindromic subsequence) probleminde bir $x[1..n]$ dizisi ($n \geq 1$) verilir ve içindeki en uzun palindrom aranır. Ancak palindromun karakterlerinin arka arkaya gelmesi gerekmez. Palindrom, baştan ve sondan okunduğunda aynı olan dizidir.

Tahtadaki örnek "character" kelimesidir (ekrandan, 01:30:00). Her karakter tek başına 1 uzunluklu bir palindromdur; bu yüzden cevap en az 1'dir. c, a, r, a, c karakterleri seçilirse "carac", yani 5 uzunluklu bir palindrom elde edilir. "ara" da bir palindromdur. Bunların en uzunu aranır.

7.2 Özyinelemeli Çözüm

$L(i, j)$, $i \leq j$ şartıyla $x[i..j]$ içindeki en uzun palindrom alt dizinin uzunluğudur; ilk aşamada palindromun kendisi değil yalnız uzunluğu bulunur. Tahtadaki sözde kod (ekrandan, 01:40:00):

```
def L(i, j):
    if i == j: return 1
    if x[i] == x[j]:
        if i + 1 == j: return 2
        else: return 2 + L(i+1, j-1)
    else:
        return max(L(i+1, j), L(i, j-1))
```

Akış şöyledir:

- Özyinelemeli fonksiyonlarda önce sonlanma noktası iyi belirlenmelidir. $i = j$ ise tek karakterlik, 1 uzunluklu bir palindrom bulunmuştur; 1 döndürülür.
- $i \neq j$ ise $x[i]$ ile $x[j]$'nin aynı olup olmadığına bakılır. Aynıysa iki durum vardır:

  - Yan yanaysalar ($i + 1 = j$), örneğin dizide "bb" ya da "aa" geçiyorsa palindrom uzunluğu 2'dir.
  - Yan yana değillerse, baştaki ve sondaki iki aynı karakter toplam uzunluğa 2 ekler ve aradaki dizide aramaya devam edilir: i bir artırılır, j bir azaltılır.
- $x[i]$ ile $x[j]$ farklıysa iki ayrı alt problemle devam edilir: i artırılıp j'ye kadar olan kısım ya da i'den $j-1$'e kadar olan kısım taranır. Hangisi daha uzun palindrom getiriyorsa o döndürülür.

7.3 Naif Çözümün Analizi

En kötü durum, dizideki karakterlerin hepsinin farklı olmasıdır; o zaman en küçük parçalara kadar inilir. Tahtadaki örnek dizi ABCDEF'tir, indisler 0..5'tir; en uzun palindrom alt dizisinin uzunluğu 1'dir (ekrandan, 01:38:20).

İlk çağrı baş ve son için, $(0, 5)$ ile yapılır. Hiçbir eşleşme bulunmadığı için $(1, 5)$ ve $(0, 4)$ çağrılır. $(1, 5)$, $(2, 5)$ ve $(1, 4)$'ü; $(0, 4)$, $(1, 4)$ ve $(0, 3)$'ü çağırır. Her seferinde ya i artar ya j azalır, diğeri sabit kalır. Sonraki seviyede $(3, 5)$, $(2, 4)$, $(2, 4)$, $(1, 3)$, ..., $(1, 3)$, $(0, 2)$ gelir; en altta $(4, 5)$, $(3, 4)$, $(2, 3)$, $(1, 2)$, $(0, 1)$ gibi yan yana çiftlere inilir ve dal orada sonlanır. $(1, 4)$, $(2, 4)$, $(1, 3)$ gibi pek çok çift tekrar eder.

Ağaç olarak çizildiğinde seviyelerdeki düğüm sayısı 1, 2, 4, 8, ... diye ikiye katlanır. Hoca "eleman sayım 5 ise ağaç seviyem 4" dedi ve tahtaya $n = 5$, $n - 1 = 4$, $\sum 2^i$ ve sonuç olarak $\Theta(2^{n-1})$ yazdı. Karmaşıklık üsteldir (eksponansiyel).

> **Ders sonrası not:** Tahtadaki dizi ABCDEF altı karakterlidir; tahtadaki $n = 5$ son indistir. Yukarıdaki kodla, bütün karakterleri farklı altı karakterli bir dizide toplam 63 çağrı yapılır (python ile sayıldı). Hangi n tanımı kullanılırsa kullanılsın sonuç üsteldir.

7.4 Memoization ile Çözüm

Üstel karmaşıklığın nedeni, koda hiçbir memoization eklenmemiş olmasıdır: $(1, 4)$ gibi değerler tekrar tekrar hesaplanır. Özyinelemeden önce şöyle bir kontrol eklenir (tahtada, ekrandan, 01:40:00): "look at $L[i, j]$ and don't recurse if $L[i, j]$ computed". Bir arama tablosu (lookup table) tutulur, önce ona bakılır; değer hesaplandıysa boşuna özyineleme yapılmaz.

Bu kontrolün işe yaraması için tablonun doldurulması gerekir. Her sonuç döndürülürken, yani her `return` satırında, $L[i, j]$ o değerle doldurulur.

Karmaşıklık yine "alt problem sayısı × alt problem başına süre" ile hesaplanır (ekrandan, 01:42:10). Her alt problem için harcanan süre sabittir, $O(1)$. Alt problem sayısı karmaşıklığı belirler: alt problemler $i < j$ olan durumlar için üretilir, yani i, j ikilileri kadar alt problem vardır:

$$\binom{n}{2} \Rightarrow \Theta(n^2)$$

Memoization, üstel karmaşıklıktan karesel karmaşıklığa inmeyi sağlar.

8. Çubuk Kesme (Rod Cutting) Problemi

8.1 Problem

Hoca öğrencilere çubuk kesme problemini daha önce görüp görmediklerini sordu; gören olmadığı için baştan anlattı. n uzunluklu bir çubuk vardır. Çubuk parçalara bölünüp satılacaktır. Her i uzunluğu için belirlenmiş bir fiyat p_i bir tabloda verilir. Amaç en yüksek geliri (revenue) elde etmektir. Bu bir optimizasyon problemidir.

- Parça uzunlukları tam sayıdır. Örneğin 5 metrelik çubuk için 1, 2, 3, 4 ve 5 metrelik satışların fiyatları verilir; yarım metreden kesilmez.
- Kesim sayısı 0 ile $n - 1$ arasındadır (tahtada "0, $n - 1$, #cuts"). Bu, kesimlerin uzunluğu değil sayısıdır. Hiç kesmeden satılabilir.

- Kesmenin bir maliyeti yoktur.

Tahtadaki fiyat tablosu (ekrandan, 01:50:00):

Uzunluk i	1	2	3	4	5	6	7	8
Fiyat p_i	1	5	8	9	10	17	17	20

Hoca, fiyatlar arasında her zaman iki kat, üç kat gibi doğrudan bir ilişki olmayabileceğini; örneğin 6 ile 7 uzunluğun fiyatının aynı (17) olabileceğini belirtti.

8.2 4 Uzunluklu Çubuk

Çubuk 4 uzunluklu olsaydı tablonun ilk dört sütunu kullanılırdı. Tahtadaki bütün seçenekler (ekrandan, 01:50:00):

Kesim uzunlukları	Gelir
4 (kesimsiz)	9
1, 3	1 + 8 = 9
2, 2	5 + 5 = 10
3, 1	8 + 1 = 9
1, 1, 2	1 + 1 + 5 = 7
1, 2, 1	7
2, 1, 1	7
1, 1, 1, 1	4

En iyi seçenek 2 + 2'dir, gelir 10'dur. 4 uzunluklu bir çubuk için bunu gözle bulmak kolaydır; ama uzunluk 8 ya da 10 olunca dinamik programlama gerekir.

Soru: Bütün seçenekler kaba kuvvetle denenecekse kaç farklı seçenek vardır? **Cevap:** Her birim (metre, inç, birim neyse) sonrasında ya bir kesim vardır ya yoktur; iki seçenek. Kesim atılabilecek $n-1$ nokta olduğu için 2^{n-1} seçenek vardır. $n=4$ için bu 8'dir ve tabloda gerçekten 8 seçenek vardır (python ile doğrulandı). Kaba kuvvet yaklaşımı üstel olarak artar.

8.3 En İyi Gelir Tablosu

Fiyat p_i o parçanın ücretidir; toplam gelir ise parçaların hepsi satıldığında elde edilen paradır ve r ile gösterilir. Hoca önce tabloyu 8 uzunluklu tek bir çubuk için değil, artan uzunluklar için kurduğunu açıkladı: problem alt problemlere ayrılarak çözüleceği için tablo da alt problemler gözetilerek oluşturulur (ekrandan, 01:55:50).

i	r_i	Nereden kesilir
1	1	1 (kesimsiz)
2	5	2 (kesimsiz)
3	8	3 (kesimsiz)
4	10	2 + 2
5	13	2 + 3
6	17	6
7	18	1 + 6 ya da 2 + 2 + 3
8	22	2 + 6

Satırların nasıl bulunduğu:

- $r_1 = 1$: tek parça satılır, kesim yoktur.
- r_2: ya 1 + 1'den kesilir (gelir 2) ya kesilmez (gelir 5). 5 seçilir.
- r_3: 1 + 2 ya da 2 + 1 (gelir 6) ya da 3 + 0 (gelir 8). 8 seçilir, kesim yoktur.
- r_4: 2 + 2, 1 + 3, 4 + 0 seçeneklerinden en fazla getiren 2 + 2'dir, gelir 10.

- $r_6 = 17$: hiç kesim yapmadan elde edilir.
- $r_7 = 18$: 1 + 6 ($1 + 17$) ve 2 + 2 + 3 ($5 + 5 + 8$) aynı geliri getirir.
- $r_8 = 22$: 2 ve 6 uzunluklu iki parça satılarak ($5 + 17$).

$r_5 = 13$ satırı tahtada yazılıdır ($5 + 8$); altyazıda ayrıca anlatılmadı. Tablo değerleri python ile doğrulandı.

8.4 Genel Bağıntıya Ulaşmak

Bu tabloda alttaki değerler, yani üstteki küçük alt problemler kullanılarak hesaplanır. Hoca r_2 ve r_3'ün nasıl bulunduğunu yazdı (ekrandan, 02:00:00):

$$r_2 = \max(p_2,\ p_1 + p_1)$$

$$r_3 = \max(p_3,\ p_1 + p_2,\ p_2 + p_1,\ p_1 + p_1 + p_1)$$

$p_1 + p_2$ ile $p_2 + p_1$ aynıdır. Ayrıca p_1, p_2 değerleri önceki adımlarda zaten hesaplanmıştır. Bu gözlemle fiyatlardan gelirlere geçilerek formül genelleştirilir:

$$r_k = \max(p_k,\ r_1 + r_{k-1},\ r_2 + r_{k-2},\ \ldots,\ r_{k-1} + r_1)$$

Örneğin $k = 4$ için parantezin içi p_4 (çubuğun bütün olarak satılması), $r_1 + r_3$ (1 uzunluklu çubuğun getireceği toplam gelir artı 3 uzunluklu çubuğun getireceği toplam gelir), $r_2 + r_2$ ve $r_3 + r_1$'dir. $p_1 + p_3$ yerine $r_1 + r_3$ yazılabilir.

Hoca sonra $r_i + r_{k-i}$ ifadesinin $p_i + r_{k-i}$'ye çevrilip çevrilemeyeceğini sordu ve çevrilebileceğini gösterdi: ilk parça kesilip fiyatıyla satılır, geri kalan kısmın en iyi geliri eklenir. Böylece

$$r_k = \max_{1 \le i \le k} \left(p_i + r_{k-i}\right), \quad r_0 = 0$$

elde edilir. Tablo artık şöyle yazılır (ekrandan, 02:10:00):

- $r_1 = p_1 + r_0$
- $r_2 = \max(p_1 + r_1,\ p_2 + r_0)$
- $r_3 = \max(p_1 + r_2,\ p_2 + r_1,\ p_3 + r_0)$
- $r_4 = \max(p_1 + r_3,\ p_2 + r_2,\ p_3 + r_1,\ p_4 + r_0)$

Örneğin r_4 için: 1 uzunluklu parçanın fiyatı ile 3 uzunluklu çubuğun en iyi geliri toplanır, bunun gibi diğer seçenekler de toplanır ve en büyüğü alınır.

Bu tabloya bakınca r_4'ü hesaplamak için r_3, r_2, r_1'in gerektiği; r_3'ü hesaplarken de r_2 ve r_1'in kullanıldığı; r_2'yi hesaplarken de r_1'in kullanıldığı görülür. Tekrar eden bir yapı vardır; memoization bu tekrarlardan kurtulacak şekilde kurulmalıdır.

9. Çubuk Kesmenin Naif Özyinelemeli Çözümü

9.1 Kod

Genel bağıntı doğrudan özyinelemeli bir fonksiyona dönüşür. Tahtadaki kod (ekrandan, 02:10:00):

```
Cut-Rod(p, n)
    if n = 0 then return 0
    end if
    q = MinInt
    for i in 1..n loop
        q = max(q, p(i) + Cut-Rod(p, n-i))
    end loop
    return q
```

$n = 0$ ise 0 döndürülür. q, yerel maksimumu bulmak için en küçük tam sayıyla (MinInt) ilklendirilir. i uzunluklu parça kesilirse onun fiyatı alınır ve geriye kalan $n - i$ uzunluklu kısmın geliri hesaplanmaya devam edilir; ikisi toplanır.

9.2 Çağrı Ağacı

Hoca fonksiyonu kısaca CR diye gösterip $CR(p, 4)$ için çağrıları tahtaya yazdı (ekrandan, 02:10:00 ve 02:16:25):

- $CR(p, 4)$, $i = 1..4$: $i = 1$ için $CR(p, 3) + p_1$; $i = 2$ için $CR(p, 2) + p_2$; $i = 3$ için $CR(p, 1) + p_3$; $i = 4$ için $CR(p, 0) + p_4$.
- $CR(p, 3)$, $i = 1..3$: $CR(p, 2) + p_1$, $CR(p, 1) + p_2$, $CR(p, 0) + p_3$. Yani 3 kez fonksiyon çağrısı yapar: 2, 1 ve 0 için.
- $CR(p, 2)$, $i = 1..2$: $CR(p, 1) + p_1$ ve $CR(p, 0) + p_2$. $CR(p, 0)$ 0 döndürdüğü için ikincisi $0 + p_2$ olur.
- $CR(p, 1)$, $i = 1$: $CR(p, 0) + p_1$.

Sonlanma $n = 0$ durumuyla olur. $CR(p, 1)$'den p_1 bulunur, yukarı döner; $CR(p, 2)$'de $p_1 + p_1$ ile p_2 karşılaştırılır, büyüğü yukarı gider; bu şekilde özyinelemeli kollar toplanarak en sonunda $CR(p, 4)$ sonlanır. Ağaçta 4'ün altında 3, 2, 1, 0; 3'ün altında 2, 1, 0; 2'nin altında 1, 0 düğümleri vardır ve henüz memoization olmadığı için 2 gibi değerler birkaç kez çağrılır. Hoca bu kısmı özyinelemeli fonksiyonların doğasını göstermek için ayrıntılı çizdiğini söyledi.

9.3 Karmaşıklık

Performans, ağacın düğüm (çağrı) sayısına bakılarak hesaplanır. Her çağrıda bir işlem (toplama) yapılır ve 1 ile n arasındaki her i için $n-i$ girdisiyle özyinelemeli çağrı yapılır (ekrandan, 02:16:25):

$$T(n) = 1 + \sum_{i=1}^{n} T(n - i), \qquad T(0) = 0$$

Değerler:

- $T(0) = 0$
- $T(1) = 1 + T(0) = 1$
- $T(2) = 1 + T(1) + T(0) = 2$
- $T(3) = 1 + T(2) + T(1) + T(0) = 4$

Değerler arasındaki ilişkiden $T(n) = 2^{n-1} = 2^n \cdot \frac{1}{2}$ bulunur; karmaşıklık üsteldir. Değerlerin $0, 1, 2, 4, 8, 16, \ldots$ diye ilerlediği python ile doğrulandı.

> **Ek bilgi (derste anlatılmadı):** Bazı ders kitaplarında aynı yineleme $T(0) = 1$ başlangıcıyla yazılır ve $T(n) = 2^n$ bulunur. Fark yalnız $n = 0$ çağrısının sayılıp sayılmamasından gelir; iki durumda da naif çözüm üsteldir.

10. Memoized ve Aşağıdan Yukarı Çubuk Kesme

Hoca öğrencilere ara isteyip istemediklerini sordu; ara verilmeden problem bitirildi.

10.1 Memoized (Özyinelemeli) Sürüm

Soru: Naif koda memoization eklemek için ne gerekir? **Cevap:** Alt problemlerin sonuçlarını tutacak, önceden olmayan bir diziye ihtiyaç vardır.

Tahtadaki kod (ekrandan, 02:20:00 ve 02:30:00):

```
Memorized-Cut-Rod(p, n)
    r : array(0..n)          -- r(0) = 0, diğerleri MinInt
    return Memorized-Cut-Rod-Aux(p, n, r)
```

```
Memorized-Cut-Rod-Aux(p, n, r)
    if r(n) = MinInt then
        q = MinInt
        for i in 1..n loop
            q = max(q, p(i) + Memorized-Cut-Rod-Aux(p, n-i, r))
        end loop
    end if
    r(n) = q
    return r(n)
```

- r dizisi $0..n$ aralığındadır. İlk gözü 0 yapılır, çünkü $T(0) = 0$'dı; diğer gözler eksi sonsuz diye düşünülebilecek en küçük tam sayıyla (MinInt) ilklendirilir.
- Aux fonksiyonu çağrıldığında önce n uzunluklu çubuk için hesaplama yapılıp yapılmadığı kontrol edilir. $r(n)$ MinInt'e eşitse henüz hesaplama yapılmamıştır; hesaplanıp doldurulması gerekir.
- Yerel maksimum bulunacağı için q MinInt ile ilklendirilir. Hoca önce "lokal minimum" deyip "lokal maksimumu bulmak için minimumla ilklendirmem gerekiyor" diye düzeltti.
- Döngü önceki çözümle aynıdır; tek fark, fonksiyona r parametresinin eklenmiş olmasıdır. $i = 1, 2, 3, ...$ için kesilen parçanın fiyatı ile geri kalan kısmın geliri toplanır; bunların en büyüğü, n uzunluklu çubuk için elde edilebilecek gelirdir ve q'da tutulur.
- `if`'ten çıkınca bulunan q, $r(n)$'ye yazılır ve döndürülür.

  Ders sonrası not: Tahtadaki yazımda `r(n) = q` satırı `end if`'ten sonra durduğu için $r(n)$ zaten hesaplıysa tanımsız bir q yazılmış olur. Doğrusu, `r(n) = q` atamasının `if` bloğunun içinde, döngüden sonra yapılmasıdır; böylece daha önce hesaplanmış değer doğrudan döndürülür.

10.2 Aşağıdan Yukarı (Bottom-Up) Sürüm

Soru: Bu özyinelemeli çözüm aşağıdan yukarıya nasıl çevrilir? **Cevap:** Özyinelemenin olduğu satırın yerine ek bir döngü mekanizması gerekir.

Tahtadaki kod (ekrandan, 02:27:20):

```
Bottom-Up-Cut-Rod(p, n)
    r : array(0..n)
    r(0) = 0
    for j in 1..n loop
        q = MinInt
        for i in 1..j loop
            q = max(q, p(i) + r(j-i))
        end loop
        r(j) = q
    end loop
    return r(n)
```

Değerleri tutan dizi yine gereklidir; bu sürümde ona DP tablosu denir ve ilk değeri 0 ile ilklendirilir. Dış döngü 1 ile n arasındaki her çubuk boyu için bir kez çalışır; amacı memoized sürümdeki döngüyle aynıdır. Özyinelemeli çağrı yapılamadığı için içte yeni bir döngü gerekir; bu kez j baz alınır. i'nin 1 ile j arasındaki değerleri için toplam boyu j olan bütün seçenekler denenir ve j boyundaki çubuk için en yüksek gelir bulunur; bulunan değer tekrar gerekirse diye saklanır. Tahtadaki `return r(n)` satırı karede görünmüyor; slayttaki genişletilmiş sürüm r'yi döndürür.

> **Ders sonrası not:** Tahtada dış döngü "for j in i..n loop" diye yazılmış (ekrandan, 02:27:20); doğrusu slayttaki gibi "for j in 1..n"dir (ekrandan, 02:34:00).

10.3 Karmaşıklık

Soru: Problemin özyinelemeli ya da aşağıdan yukarı çözülmesi karmaşıklığı etkiler mi? **Cevap:** Etkilemez.

Aşağıdan yukarı sürümde iç içe iki döngü vardır; dış döngü $j = 1..n$, iç döngü $i = 1..j$ gider ve içeride sabit bir işlem yapılır (ekrandan, 02:30:00):

$$\sum_{j=1}^{n} \sum_{i=1}^{j} 1 = \sum_{j=1}^{n} j = \frac{n(n+1)}{2} = \Theta(n^2)$$

Memoized sürümde formül yine "alt problem sayısı × her alt problemi çözmek için gereken süre"dir (tahtada "#subproblems × time required for solving each subproblem", ekrandan, 02:31:20). Her $r(n)$ ataması bir alt problem çözümüdür; n alt problem vardır. Her alt problem için dönme sayısı değişir (4 için 4, 5 için 5 gibi), ama genel olarak n denebilir. Sonuç yine $O(n^2)$'dir. Alt problem sayısı ve her alt problem için dönme sayısı belli olduğu için memoized sürümü özyinelemeli ya da aşağıdan yukarı yazmak sonucu değiştirmez.

11. Kesim Yerlerini Bulmak

Şimdiye kadarki algoritmaların hiçbiri en yüksek geliri getiren kesim noktalarını söylemez; yalnız en yüksek geliri verir. Kesimlerin nerede yapılacağını bulmak için bu bilgiyi tutacak ek bir dizi gerekir. Kod biraz daha uzun olduğu için hoca onu tahtaya yazmadı, slayttan gösterdi (ekrandan, 02:34:00; slayt başlığı "Rod Cutting: Finding the Solution, Memorized bottom up (non-recursive)"):

```
ExtendedBottomUpCutRod(p, n)
    r: array(0..n)    -- optimal value for rods of length 0..n
    s: array(0..n)    -- optimal first cut for rods of length 0..n
    r(0) := 0
    for j in 1 .. n loop
        q := MinInt
        for i in 1 .. j loop   -- Find the max cut position for length j
            if q < p(i) + r(j-i) then
                q := p(i) + r(j-i)
                s(j) := i      -- Remember the value of best so far value of i
            end if
        end loop
        r(j) := q
    end loop
    return r and s

PrintCutRodSolution(p, n)
    (r, s) := ExtendedBottomUpCutRod(p, n)
    while n > 0 loop
        print s(n)
        n := n - s(n)
    end loop
```

Döngüler aşağıdan yukarı sürümle aynıdır; maksimum `max` yerine `if` ile bulunur. Her seferinde j'ye kadar gidilip en yüksek getiren seçenek taranır; formül yine i uzunluklu parçanın fiyatı artı geri kalan kısmın geliridir. Fark şudur: daha iyi bir seçenek bulunduğu her noktada o i, yani ilk kesimin yeri $s(j)$'ye yazılır. Çözümü yazdırırken $s(n)$ basılır, çubuktan o kadar kesilir ve geri kalan için aynı işlem tekrarlanır.

Zaman karmaşıklığı değişmez; ek dizi yüzünden bellek (space) karmaşıklığı artar.

> **Ek bilgi (derste anlatılmadı):** Derste çözülen fiyat tablosuyla bu algoritma $n = 1..8$ için $s = (1, 2, 3, 2, 2, 6, 1, 2)$ verir; $n = 8$ için yazdırılan kesimler 2 ve 6, $n = 4$ için 2 ve 2'dir. Bu değerler, yukarıdaki en iyi gelir tablosuyla uyumludur (python ile hesaplandı).

Hoca, bununla ilgili bir problem daha olduğunu ama uzun olduğu için o derste geçmeyeceğini söyledi. Sırt çantası (knapsack) problemi daha sık görüldüğü için onu çözmeyeceğini de belirtti.

Örnek Sorular

Soru 1 (derste sözlü, 00:23:11; ekrandan, 00:36:30). h hem tek yönlü hem TCR olsun. h'yi kullanarak hâlâ tek yönlü olan ama çakışmaya dayanıklı olmayan bir h' fonksiyonu kurun.

> **Çözüm (derste verildi):** İlk girdi biti yerine iki yeni bitin XOR'u verilir: $h'(a, b, x_2, \ldots, x_n) = h(a \oplus b, x_2, \ldots, x_n)$. Diğer bitler değişmediği için tek yönlülük korunur. $h'(1, 1, x_2, \ldots, x_n) = h'(0, 0, x_2, \ldots, x_n)$ olduğundan çakışma bulmak kolaydır.

Soru 2 (derste sözlü, 00:31:46; ekrandan, 00:36:30). Aynı h'yi kullanarak tek yönlülüğü bozan ama TCR'yi koruyan bir fonksiyon tasarlayın.

> **Çözüm (derste verildi):** $h''(x) = 0 \,\|\, x$ ($|x| \leq n$ ise), aksi hâlde $1 \,\|\, h(x)$. Kısa girdilerde çıktıdaki baştaki 0 silinerek x bulunur, tek yönlülük bozulur. Başı 0 olan çıktılar girdiler farklı olduğu için farklıdır; başı 1 olanlar h'nin TCR özelliğini taşır; TCR korunur.

Soru 3 (derste sözlü, 00:39:34, 00:49:21, 00:55:20 ve 01:05:26; ekrandan, 00:50:00, 01:00:00 ve 01:07:20). Parola saklama, dosya değişikliği algılayıcı, dijital imza ve taahhüt (kapalı zarf ihale) uygulamalarında OW, CR/TCR ve NM özelliklerinden hangileri önemlidir?

> **Çözüm (derste verildi):** Parola saklama: en önemlisi OW (saklanan $h(PW)$'den parolaya dönülmemeli); CR/TCR de gerekli ama uzun parolada çakışma ihtimali düşüktür. Dosya değişikliği: OW önemli değildir (dosya açıktır); $h(F) = h(F')$ olan F' üretilememelidir (TCR/CR). Dijital imza: $h(M) = h(M')$ olan M' bulunamamalıdır (TCR/CR); belgeler arasında x, $x + 1$ gibi ilişki yoksa NM önemli değildir. Taahhüt: OW ($C(x)$, x'i açığa çıkarmamalı), CR (teklif veren aynı hash'i veren başka bir teklif iddia edememeli) ve NM ($C(x)$'ten $C(x + 1)$ üretilememeli) birlikte önemlidir.

Soru 4 (derste sözlü, 01:41:27; ekrandan, 01:42:10). En uzun palindrom alt dizisinin memoized çözümünde kaç alt problem vardır ve karmaşıklık nedir?

> **Çözüm (derste verildi):** Alt problemler $i < j$ olan (i, j) çiftleridir: $\binom{n}{2}$ tane. Her biri $O(1)$ sürede çözülür. Toplam $\Theta(n^2)$; memoization olmadan üsteldir.

Soru 5 (derste sözlü, 01:49:50; ekrandan, 01:50:00). Fiyatları $p_1..p_4 = 1, 5, 8, 9$ olan tabloda 4 uzunluklu çubuğun bütün kesim seçeneklerini ve gelirlerini yazın. n uzunluklu bir çubukta kaç seçenek vardır?

> **Çözüm (derste verildi):** 4 → 9; 1,3 → 9; 2,2 → 10; 3,1 → 9; 1,1,2 → 7; 1,2,1 → 7; 2,1,1 → 7; 1,1,1,1 → 4. En iyisi 2 + 2, gelir 10. Her birim noktada kesim ya vardır ya yoktur; 2^{n-1} seçenek, $n = 4$ için 8.

Soru 6 (ekrandan, 01:55:50 ve 02:00:00). Uzunluk 1..8 için fiyatlar 1, 5, 8, 9, 10, 17, 17, 20'dir. $r_1, \ldots, r_8$ en iyi gelirlerini ve kesim yerlerini bulun. Genel bağıntıyı yazın.

> **Çözüm (derste verildi):** $r_k = \max_{1 \leq i \leq k}(p_i + r_{k-i})$, $r_0 = 0$. $r_1 = 1$ (kesimsiz), $r_2 = 5$ (kesimsiz), $r_3 = 8$ (kesimsiz), $r_4 = 10$ (2 + 2), $r_5 = 13$ (2 + 3), $r_6 = 17$ (6), $r_7 = 18$ (1 + 6 ya da 2 + 2 + 3), $r_8 = 22$ (2 + 6).

Soru 7 (ekrandan, 02:16:25). Naif `Cut-Rod` için çağrı sayısı yinelemesini yazın ve çözün.

> **Çözüm (derste verildi):** $T(n) = 1 + \sum_{i=1}^{n} T(n-i)$, $T(0) = 0$. $T(1) = 1$, $T(2) = 2$, $T(3) = 4$; genel olarak $T(n) = 2^{n-1}$, üstel.

Soru 8 (derste sözlü, 02:24:05 ve 02:27:36; ekrandan, 02:27:20 ve 02:30:00). Memoized özyinelemeli `Cut-Rod`'u aşağıdan yukarı sürüme çevirin. Bu çevirme karmaşıklığı değiştirir mi?

> **Çözüm (derste verildi):** $r(0) = 0$; `for j in 1..n`: $q =$ MinInt; `for i in 1..j`: $q = \max(q, p(i) + r(j - i))$; sonra $r(j) = q$. Özyinelemeli çağrının yerini iç döngü alır. Süre $\sum_{j=1}^{n} j = n(n + 1)/2 = \Theta(n^2)$; memoized özyinelemeli sürüm de n alt problem × alt problem başına en fazla n dönüş $= O(n^2)$. Karmaşıklık değişmez.

Dersin Toparlaması

Dersin ilk yarısında hash fonksiyonlarına hash tablosu bağlamının dışından, uygulama gözüyle bakıldı. Değişken uzunluklu girdiyi sabit uzunluklu çıktıya dönüştüren bir fonksiyondan istenebilecek beş özellik tanımlandı: tek yönlülük, çakışmaya dayanıklılık, hedefli çakışmaya dayanıklılık, sözde rastgelelik ve biçimlendirilemezlik. CR'nin TCR'yi gerektirdiği, ama tek yönlülükle çakışmaya dayanıklılık arasında böyle bir bağ olmadığı XOR ve $0 \| x$ karşı örnekleriyle gösterildi. Ardından parola saklama, dosya değişikliği algılama, dijital imza ve kapalı zarf ihale (commitment) uygulamalarında hangi özelliğin neden önemli olduğu tartışıldı. Ana fikir, özelliklerin öneminin uygulamaya bağlı olmasıdır.

İkinci yarıda dinamik programlamaya geçildi. Örtüşen alt problemleri bir kez çözüp saklamanın (memoization ya da DP tablosu) üstel çözümleri polinom süreye indirdiği üç örnekle gösterildi: Fibonacci ($O(n)$), en uzun palindrom alt dizisi ($\Theta(n^2)$) ve çubuk kesme ($\Theta(n^2)$). Çubuk kesmede $r_k = \max(p_i + r_{k-i})$ bağıntısı adım adım kuruldu; naif, memoized ve aşağıdan yukarı sürümler yazıldı ve kesim yerlerini veren genişletilmiş sürüm gösterildi. Karmaşıklığın "alt problem sayısı × alt problem başına süre" ile hesaplandığı ve özyinelemeli ya da aşağıdan yukarı yazmanın bunu değiştirmediği vurgulandı. Hoca, parola uzunluğu ile çakışma ilişkisine dair soruyu sonraki derste bir örnekle açmayı önerdi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.